DBT • CLASS NOTES

Cursors & Error Handling

DBT Day 9 — 17 October

Complete, visual notes from today's lecture: what a cursor really is, the declare → open → fetch → close lifecycle, the salary-increment and archive examples, then error handling — handlers, error codes vs SQLSTATE, SIGNAL, RESIGNAL and rollback — plus the full exam briefing.

LEARN BY DOING

Study notes • prepared 17 October 2026
Topics: cursors • error handling | MySQL 8.0

Table of Contents

Day 9 at a Glance 📌

- 🎯 What we covered today
- 🗃️ The tables we used in class
- 🧭 The shape of today
- 🔑 The three sentences that carry the whole day

Part A — Cursors 🎞️

- 🎞️ What is a cursor?
- 🧐 Why do we need a cursor? (set-based SQL vs cursor)
- 🔁 The lifecycle of a cursor
- 🚨 Declaration order matters
- 🛑 The NOT FOUND handler — how the loop knows it is done
- 💉 Example 1 — salary increment by grade (apply_salary_increment)
- 🔄 Same cursor, different loops
- 🗄️ Example 2 — archiving low-salary employees (archive_low_salary)
- 🆚 Cursor vs temporary table
- ✅ When to use a cursor — and when to avoid it
- ✅ Part A recap

Part B — Error Handling 🛡️

- 🧭 What happens with and without a handler
- 🔢 MySQL error code vs SQLSTATE
- 🧱 Declaring a handler
- 📃 The condition types you can handle
- 💉 Example — a friendly message for duplicate keys (add_student)
- 🏷️ Named conditions
- 📣 Raising your own error — SIGNAL
- ↩️ RESIGNAL — handle, act, then re-throw
- 🌐 The general error handler (SQLEXCEPTION)
- 💳 Transactions + error handling (roll back on failure)
- 🧩 Where you can use error handling
- ✅ Part B recap

Part C — Cheat Sheet, Gotchas & Practice 🏁

- 📋 Cheat sheet
- 🐞 Errors and gotchas we actually hit today
- 🧠 Practice exercises (do these — they mirror the class)
- 🧩 Solutions
- 🎓 Final words

Part D — Exam Briefing & What's Next 📝

- 🗓️ The schedule (Monday)
- 🖥️ The environment rules (read carefully)
- 📄 The solution file — how marks are awarded
- 🧾 What the paper will ask (based on the briefing)
- ✅ Quick pre-exam checklist

Day 9 at a Glance 📌

These are your notes from **DBT Day 9 (17 Oct)** — the session that finished the practical MySQL course with **two big topics**: **cursors** and **error handling**. It ended with a full **exam briefing** from Vipul sir. This chapter is the map for everything that follows.

> 💡 **How to read these notes**
>
> Keep MySQL open next to this PDF. Every concept is followed by a query you can **type and run yourself**. Type it — do not copy-paste — and try to *predict* the output before you press `Enter`. That is how a lecture turns into a skill.

🎯 What we covered today

Part	Topic	The one-line summary
A	**Cursors** 🔲	A variable that **fetches the rows of a query one at a time** so you can process each row
B	**Error Handling** 🛡️	Telling a stored program **what to do when an error happens** (continue, exit, log, roll back)
C	Cheat sheet & practice	Compressed recap, gotchas and exercises
D	Exam briefing 📝	Command-line rules, solution-file format, schedule

We also spent time on **the two full cursor examples** (salary increment by grade, and archiving low-salary employees) and on `SIGNAL` / `RESIGNAL` — the mechanism behind the "salary cannot be negative" rule from Day 8.

> ℹ️ **Where we are in the course**
>
> - **Day 7:** stored **procedures** — reusable programs you call with `CALL`.
> - **Day 8:** stored **functions** + **triggers**.
> - **Today (Day 9): cursors** and **error handling** — the last MySQL topics. 🏁

> ℹ️ **Still pending (not today)**
>
> The teacher confirmed three topics are still left and will be taken later (possibly after the exam / on the first day of your **WPT** module on Wednesday):
>
> - **Normalization** (≈ 1 hour)
> - **NoSQL**
> - **MongoDB** *(explicitly NOT in the MySQL exam)*

🗂️ The tables we used in class

We reused the `employee` table (adding a `grade` column) and created an archive table, plus the `student` table for the error-handling demo. Run this once so every example in these notes works.

```
-- ------------------------------------------------------------------------
-- The employee table — reused from earlier days, with a new "grade" column
-- ------------------------------------------------------------------------
CREATE TABLE employee (
  emp_id   INT PRIMARY KEY,
  emp_name VARCHAR(30),
  salary   DECIMAL(10,2),
  grade    VARCHAR(2)          -- 'A' / 'B' / 'C' (A+ is possible -> 2 chars)
);

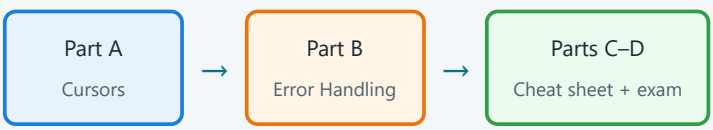
INSERT INTO employee (emp_id, emp_name, salary, grade) VALUES
  (101, 'Amit Patel', 75000, 'B'),
  (102, 'Priya',      55000, 'C'),
  (103, 'Rahul',      98000, 'A'),
  (104, 'Rohit',      60000, 'B'),
  (105, 'Aniket',     25000, 'C'),
  (106, 'Raj',        30000, 'C');

-- ------------------------------------------------------------------------
-- Created TODAY: the archive table for the low-salary example
-- ------------------------------------------------------------------------
CREATE TABLE employee_archive (
  emp_id     INT,
  emp_name   VARCHAR(30),
  salary     DECIMAL(10,2),
  grade      VARCHAR(2),
  archive_on DATETIME          -- the moment we archived the row (NOW())
);

-- ------------------------------------------------------------------------
-- The student table used for the error-handling demo
-- ------------------------------------------------------------------------
CREATE TABLE student (
  student_id    INT PRIMARY KEY,     -- duplicate here -> error 1062
  name          VARCHAR(30),
  department_id INT,                 -- foreign key in the class DB
  marks         DECIMAL(5,2)
);
```

> ⚠️ **Table names and values may differ on your machine**
>
> If your tables use different column names, adjust the queries. The *ideas* are what matter and they are identical either way. The numbers below are the ones the teacher used in class, so the expected outputs line up.

🧭 The shape of today



Row by row, then error by error — then one big exam briefing.

🔑 The three sentences that carry the whole day

A cursor	A handler	SIGNAL
fetches rows **one at a time**, sequentially — you supply the logic	tells MySQL **what to do when a condition fires** (error, warning, no rows)	lets **you** raise your own error with your own message

Cursor = read rows. Handler = react to conditions. SIGNAL = raise your own.

> 💡 **The golden rule for cursors (said more than once in class)**
>
> **Try set-based SQL first.** Prefer `UPDATE ... WHERE`, `JOIN`, `GROUP BY`, aggregate functions, CTEs or window functions. Reach for a cursor **only** when the logic genuinely needs **row-by-row, procedural decisions**.

Ready? Let's walk a result set one row at a time. 🎞️

Part A — Cursors 🎞️

Everything in SQL so far has been **set-based**: you describe the *result you want* and MySQL produces all of it in one shot. That is the normal, correct, fast way to work.

But sometimes you must work **one row at a time** — read a row, take a decision based on *that* row, act, then move to the next. That is exactly what a **cursor** is for.

🎞️ What is a cursor?

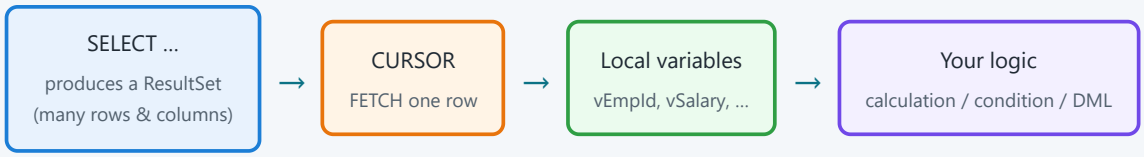
> ℹ️ **The definition to write down first**
>
> **A cursor is a special type of variable present in RDBMS which is used to fetch the rows one by one** — it processes **multiple rows, but a single row at a time**.

The important properties, in the teacher's words:

- It is a **special type of variable** — so you **declare** it like a variable.
- Its **only job is to fetch rows one by one**. The *logic* is yours to write.
- It is **based on a `SELECT` statement** — the `SELECT` produces the result set the cursor walks.
- It is **read-only** — a cursor never changes data. You pair it with `UPDATE` / `INSERT` / `DELETE` statements to act on each row.

From `SELECT` to a value in your hands

The `SELECT` you attach to a cursor produces a **result set** — many rows and columns. The cursor then hands you that result set **one row at a time**, dropping each row's columns into **local (intermediate) variables**. You use those variables for calculation, conditions, and DML.



A cursor is the pipe between a result set and your row-by-row code.

Moving in one direction only

> ⚠️ **Cursors are sequential — forward only**
>
> A MySQL stored-program cursor **moves from the first row toward the last row**, one step at a time. You **cannot normally fetch backward**. Once a row has been fetched, the cursor has moved past it.

🤔 Why do we need a cursor? (set-based SQL vs cursor)

A plain `UPDATE` is **set-based**: one statement touches every matching row at once.

```
-- Set-based: EVERY employee gets +5000, no questions asked
UPDATE employee SET salary = salary + 5000;
```

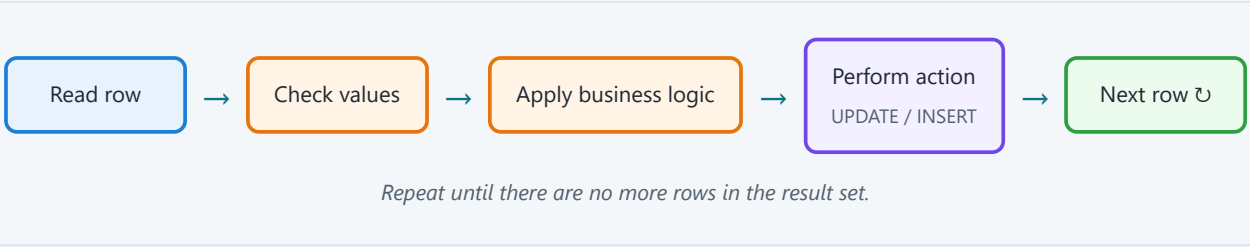
That is fine when *the same operation* applies to every row. But real payroll is not like that. In the class example, the increment depends on the employee's **salary slab** — people on lower salaries should get a *bigger* hike:

Salary slab	Typical yearly hike
below ~50,000	~10 %
50,000 – 75,000	~5 %
above ~75,000	~2–3 %

You cannot express "a different percentage for each row" with a single simple `UPDATE` — you need to **read one row, decide, act, and move on**. That is the cursor.

> ℹ️ **When a cursor becomes useful**
>
> A cursor becomes useful **when the program must read one row, make a *procedural* decision on that particular row, perform an action, and then move to the next row.**

The cursor flow



Set-based SQL vs cursor — the comparison to remember

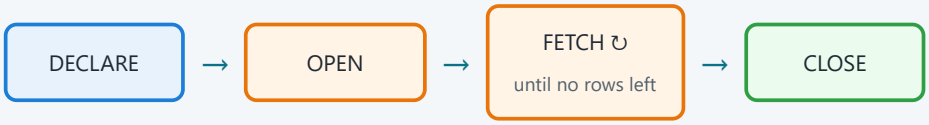
	Set-based SQL	Cursor
How many rows at a time	a **set** of rows in a single go	**one row** at a time
Good for	**bulk** operations (same action for all rows)	**procedural**, row-by-row logic
Speed	**simpler and faster**	more work; needs extra programming
Extra machinery	none	**local variables** to hold the intermediate row
Typical example	`UPDATE employee SET salary = salary + 5000`	increment **according to each row's grade**

> 💡 **The rule from class**
>
> **First try set-based SQL.** Use a cursor **only when row-by-row procedural processing is really required**. In industry, if a plain SQL statement (or a `JOIN`, aggregate, CTE or window function) can solve it, **use that** and avoid the cursor.

🔁 The lifecycle of a cursor

Every cursor follows the same four steps, plus one essential safety net (the handler):

Step	Statement	Purpose
1	`DECLARE … CURSOR FOR SELECT …`	Define which query the cursor will walk
2	`DECLARE CONTINUE HANDLER FOR NOT FOUND …`	Detect "no more rows"
3	`OPEN …`	Run the query; position before the first row
4	`FETCH … INTO …`	Read the next row into variables (**repeat in a loop**)
5	`CLOSE …`	Release the cursor



Declare → Open → Fetch (loop) → Close. Always close the cursor.

The general syntax

```
DECLARE cursor_name CURSOR FOR
    SELECT col1, col2, ...
      FROM your_table
     WHERE some_condition;          -- the cursor's result set

OPEN cursor_name;

-- loop: FETCH → process → FETCH → process → ...
FETCH cursor_name INTO var1, var2, ...;

CLOSE cursor_name;
```

> ℹ️ **Cursors live inside stored programs**
>
> A cursor is used inside a **stored procedure / function** (inside the `BEGIN … END` block). You cannot declare one at the top level of the `mysql` client.

🚨 Declaration order matters

Inside the `BEGIN … END` block, declarations **must** appear in this order:

```
1. DECLARE variables          (and conditions)
2. DECLARE cursors
3. DECLARE handlers
```

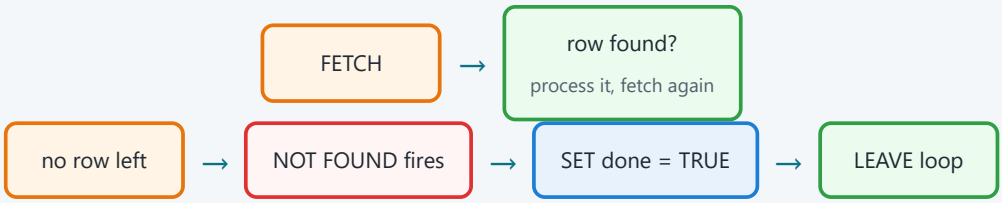
> 🛑 **Get the order wrong and you get a vague syntax error**
>
> **variables → cursors → handlers.** Memorise it. A cursor declared before the variables it fetches into, or a handler before the cursor, is a syntax error.

🛑 The `NOT FOUND` handler — how the loop knows it is done

`FETCH` fails when the cursor runs off the end of the result set, raising a `NOT FOUND` condition. We catch it with a **handler** that sets a flag, then the loop checks the flag and leaves.

```
DECLARE done BOOLEAN DEFAULT FALSE;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
```

- `CONTINUE` means: for this condition, run the handler and then **continue** with the next statement (instead of `EXIT`ing the block).
- After a failed `FETCH`, `done` becomes `TRUE`, and our loop checks it and `LEAVE`s.
- Without this handler, the cursor would not know the rows have ended.



The handler is the signal that "the rows are over" — then the loop exits.

> ℹ️ **One flag, two jobs (the point a student asked about)**
>
> The **same** `done` **variable** is used in **two places**: the handler **sets** it, and the loop **reads** it in its `IF done THEN LEAVE …` guard. The handler works *internally* — you do not see it being called; it simply flips the flag when the fetch finds no more rows.

🧪 Example 1 — salary increment by grade (`apply_salary_increment`)

Business rule: *for every employee, read their grade and give a different increment.*

Grade	Increment
`A`	**5 %** (`salary * 1.05`)
`B`	**10 %** (`salary * 1.10`)
`C`	**15 %** (`salary * 1.15`)
anything else	**13 %** (`salary * 1.13`) — the safety-net `ELSE`

```
DELIMITER $$

CREATE PROCEDURE apply_salary_increment()
BEGIN
    -- (1) local variables ---------------------------------------------
    DECLARE done      BOOLEAN DEFAULT FALSE;
    DECLARE vEmpId    INT;
    DECLARE vEmpName  VARCHAR(30);       -- fetched, though not used in the logic
    DECLARE vSalary   DECIMAL(10,2);
    DECLARE vGrade    VARCHAR(2);

    -- (2) the cursor --------------------------------------------------
    DECLARE cur_emp CURSOR FOR
        SELECT emp_id, emp_name, salary, grade
          FROM employee;

    -- (3) the handler -------------------------------------------------
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur_emp;

    salary_loop: LOOP
        FETCH cur_emp INTO vEmpId, vEmpName, vSalary, vGrade;

        IF done THEN
            LEAVE salary_loop;           -- no more rows -> stop
        END IF;

        IF vGrade = 'A' THEN
            UPDATE employee SET salary = salary * 1.05 WHERE emp_id = vEmpId;
        ELSEIF vGrade = 'B' THEN
            UPDATE employee SET salary = salary * 1.10 WHERE emp_id = vEmpId;
        ELSEIF vGrade = 'C' THEN
            UPDATE employee SET salary = salary * 1.15 WHERE emp_id = vEmpId;
        ELSE
            -- no grade matched: a precautionary branch
            UPDATE employee SET salary = salary * 1.13 WHERE emp_id = vEmpId;
        END IF;
    END LOOP;

    CLOSE cur_emp;
END $$

DELIMITER ;
```

Run it and watch it work

```
CALL apply_salary_increment();

SELECT * FROM employee;
```

Expected result (using the seed data from the overview):

emp_id	emp_name	before	grade	rule	after
101	Amit Patel	75,000	B	×1.10	**82,500**
102	Priya	55,000	C	×1.15	**63,250**
103	Rahul	98,000	A	×1.05	**102,900**
104	Rohit	60,000	B	×1.10	**66,000**
105	Aniket	25,000	C	×1.15	**28,750**
106	Raj	30,000	C	×1.15	**34,500**

The teacher's live check confirmed the first row: `Amit Patel`, grade **B** → `75,000 × 1.10 = 82,500`. ✅

> 💡 **Why `emp_name` is fetched but never used**
>
> We fetch `emp_name` into `vEmpName` only because the cursor's `SELECT` returns it and each `FETCH` must supply a variable for every selected column. The logic identifies the row by `emp_id` (`WHERE emp_id = vEmpId`), so `emp_name` is not needed. Keep the variable to match the column list.

How the loop runs (trace it once)

1. **Fetch** `101, Amit Patel, 75000, B` → `done` is `FALSE` → grade is `B` → `UPDATE` row 101 → `salary = 75000 × 1.10 = 82500`.
2. **Fetch** the next row → process it. Then the next. And so on.
3. After the **last** row, the next `FETCH` **finds nothing** → the handler sets `done = TRUE` → `IF done THEN LEAVE salary_loop` exits the loop.
4. `CLOSE cur_emp;`

🔁 Same cursor, different loops

The `FETCH` inside a **labelled** `LOOP` is the most common pattern, but a cursor also works with `WHILE` and `REPEAT`. Only the loop's shape changes — the logic in the middle is the same.

With `WHILE` (fetch once before the loop)

```
OPEN cur_emp;

FETCH cur_emp INTO vEmpId, vEmpName, vSalary, vGrade;   -- prime the first row

WHILE NOT done DO
    -- === your logic for the current row ===
    FETCH cur_emp INTO vEmpId, vEmpName, vSalary, vGrade;   -- next row
END WHILE;

CLOSE cur_emp;
```

> ℹ️ **Understanding `WHILE NOT done`**
>
> `done` starts as `FALSE`. `NOT FALSE` = `TRUE` → the loop body runs. When the rows end, the handler sets `done = TRUE`, so `NOT TRUE` = `FALSE` → the loop exits. (This is the logic the teacher walked through live, and it confused several people — read it slowly once and it clicks.)

With `REPEAT` (check at the bottom)

```
OPEN cur_emp;

REPEAT
    FETCH cur_emp INTO vEmpId, vEmpName, vSalary, vGrade;

    IF NOT done THEN
        -- === your logic for the current row ===
    END IF;
UNTIL done END REPEAT;

CLOSE cur_emp;
```

Loop	Check happens	Shape
`LOOP … END LOOP`	top (with `IF done THEN LEAVE`)	flexible, most common
`WHILE NOT done DO … END WHILE`	top	prime the first `FETCH` before the loop
`REPEAT … UNTIL done END REPEAT`	bottom	body always runs at least once

Three ways to walk the same cursor — pick one and be consistent.

🗄️ Example 2 — archiving low-salary employees (`archive_low_salary`)

Business rule: *copy every employee earning ≤ 50,000 into an archive table, stamping the moment they were archived.*

Step 1 — the archive table

```
CREATE TABLE employee_archive (
  emp_id     INT,
  emp_name   VARCHAR(30),
  salary     DECIMAL(10,2),
  grade      VARCHAR(2),
  archive_on DATETIME
);
```

> **Choose only the columns you need**
>
> The teacher pointed out that you can keep or drop columns in the archive table depending on what you need. Here we keep `grade`; in a different archive you might drop it. **Only the columns you keep need matching variables** in the procedure.

Step 2 — the procedure

```
DELIMITER $$

CREATE PROCEDURE archive_low_salary()
BEGIN
    DECLARE done      BOOLEAN DEFAULT FALSE;
    DECLARE vEmpId    INT;
    DECLARE vEmpName  VARCHAR(30);
    DECLARE vSalary   DECIMAL(10,2);
    DECLARE vGrade    VARCHAR(2);

    DECLARE cur_emp CURSOR FOR
        SELECT emp_id, emp_name, salary, grade
          FROM employee;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur_emp;

    archive_loop: LOOP
        FETCH cur_emp INTO vEmpId, vEmpName, vSalary, vGrade;

        IF done THEN
            LEAVE archive_loop;
        END IF;

        IF vSalary <= 50000 THEN
            INSERT INTO employee_archive (emp_id, emp_name, salary, grade, archive_on)
            VALUES (vEmpId, vEmpName, vSalary, vGrade, NOW());
        END IF;
    END LOOP;

    CLOSE cur_emp;
END $$

DELIMITER ;
```

Step 3 — run it

```
CALL archive_low_salary();

SELECT * FROM employee_archive;
SELECT * FROM employee;      -- the source rows are NOT changed — the cursor is read-only
```

Expected result — after the salary increment above, only the two lowest earners qualify (≤ 50,000):

```
+--------+----------+---------+-------+---------------------+
| emp_id | emp_name | salary  | grade | archive_on          |
+--------+----------+---------+-------+---------------------+
|    105 | Aniket   | 28750.00| C     | 2026-10-17 13:37:16 |
|    106 | Raj      | 34500.00| C     | 2026-10-17 13:37:16 |
+--------+----------+---------+-------+---------------------+
```

🍼 What the teacher stressed here

- The **same row is fetched into the variables for every iteration** — that is the meaning of "fetch one row at a time **into the variables**".
- The **condition** (`vSalary <= 50000`) decides whether anything is written; rows that fail it are simply skipped and the loop moves on.
- The cursor itself **never modifies** the employee table. The `INSERT` is *your* logic.

💡 `NOW()` for the timestamp

`NOW()` returns the current **date and time** — perfect for the `archive_on` `DATETIME` column. It is a built-in function (recall the date functions from the functions chapter).

🧮 Cursor vs temporary table

The teacher closed the cursor theory with this comparison:

	Temporary table	Cursor
What it holds	a **temporary set of data** you can query	a **pointer over a result set**
Access style	set-based — you query it like any table	**sequential, row by row**
Gives you	rows you can `SELECT` , `JOIN` , aggregate	exactly **one row at a time**

A temp table stores data so you can *again* work set-based; a cursor exists purely to feed you rows **in order**, one by one, so you can run procedural logic.

✅ When to use a cursor — and when to avoid it

Use a cursor when...

- **Each row requires a different procedural decision** — the action depends on that row's values (our grade-based increment).
- **Processing one row determines the sequence of actions** for that row.
- You need a **progressively constructed report or string** (building a formatted output line by line).
- The **business logic is difficult or impractical to express** with set-based SQL.

Avoid the cursor when...

If you can do it with...	Use that instead
a single `UPDATE` / `INSERT` / `DELETE` / `SELECT`	the plain statement
aggregate functions (`SUM` , `COUNT` , `AVG` , ...)	the aggregate
joins	a `JOIN` (or `UPDATE … JOIN`)
CTEs (`WITH …`)	the CTE
window functions (`SUM() OVER …` , `ROW_NUMBER() OVER …`)	the window function

> 🔴 **The industry habit**
>
> Whenever you write a cursor in a real job, first ask: "*Could a set-based query do this?*" If yes, **prefer the query** — it is simpler and faster. Cursors are for the genuinely sequential cases.

✅ Part A recap

- A **cursor fetches rows one by one** from a `SELECT` result set; it is **read-only** and **forward-only (sequential)**.
- Lifecycle: **DECLARE → OPEN → FETCH (loop) → CLOSE**.
- Declare in order: **variables → cursor → handler**.
- `DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;` + a `done` flag is what ends the loop safely.
- The same fetch-and-process logic works with `LOOP` , `WHILE` and `REPEAT` .
- Our two examples: `apply_salary_increment` (grade-based hike) and `archive_low_salary` (copy rows ≤ 50,000 to an archive).
- **Prefer set-based SQL**; use a cursor only when row-by-row logic is truly needed.

On to the second half: **error handling**. 🛡️

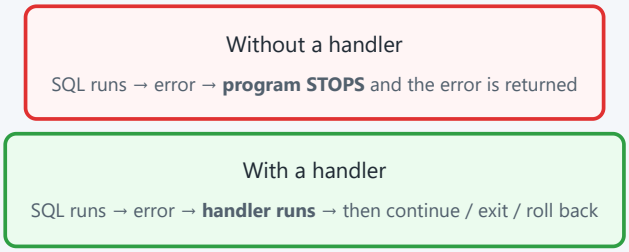
Part B — Error Handling 🛡️

A stored program that can only run when *nothing goes wrong* is a fragile program. **Error handling** is how a stored program stays in control when something does go wrong — a duplicate key, a bad value, a broken transaction.

> ℹ️ **The definition**
>
> **Error handling means detecting an error during execution and deciding what the stored program should do when that error occurs.**

We already used one handler with cursors: `DECLARE CONTINUE HANDLER FOR NOT FOUND …`. Now we generalise the idea.

🧭 What happens with and without a handler



A handler turns a hard stop into a decision.

The example we saw live: inserting a **duplicate primary key** into `student`.

```
INSERT INTO student (student_id, name, department_id, marks)
VALUES (103, 'Someone', 10, 80);          -- 103 already exists!
```

```
ERROR 1062 (23000): Duplicate entry '103' for key 'student.PRIMARY'
```

Without a handler the program stops and dumps that raw error on the user. **With a handler**, we can catch `1062` and show a friendly message instead, then decide whether to continue.

- **Without handler:** statements run → error → program **stops**, error returned.
- **With handler:** statements run → error → **handler executes** → then the program **continues**, **exits** the block, or **rolls back** (whatever the handler says).

🔢 MySQL error code vs SQLSTATE

When something goes wrong, MySQL identifies it in **two ways**:

Kind	Example	Meaning	Typical use
MySQL error code	`1062`	a **specific numeric code** for one named MySQL error	duplicate-key detection
SQLSTATE	`23000`	a **5-character standard code** (a *category*)	integrity/constraint violations
Custom SQLSTATE	`45000`	the code reserved for **user-defined** errors	your own `SIGNAL` messages

In the live demo the duplicate-key insert produced **both**: `ERROR 1062 (23000)` — the number `1062` is the MySQL error code, and `23000` is the SQLSTATE that means "integrity constraint violated" (the primary key rejected the duplicate).



1062 is exact; 23000 is the category; 45000 is yours to raise.

> ℹ️ **Where we had already used `45000`**
>
> In Day 8's trigger we raised our own error with `SIGNAL SQLSTATE '45000'` to say "*salary cannot be negative*". That is the **custom SQLSTATE** in action — a message we invented, using a code that is reserved for exactly that.

🧱 Declaring a handler

```
DECLARE {CONTINUE | EXIT} HANDLER
    FOR <condition>            -- e.g. NOT FOUND, 1062, SQLSTATE '23000', SQLEXCEPTION
    <statement>;               -- what to do when it fires
```

- The handler declaration goes **inside the `BEGIN … END` block**.
- `CONTINUE` vs `EXIT` decides what happens *after* the handler runs.

`CONTINUE` vs `EXIT`

Handler	What it does on an error	Use it when…
CONTINUE	handles the error and **continues** with the remaining statements	later statements should still run (e.g. skip one bad row, insert the rest)
EXIT	handles the error and **exits the current block**	the operation is critical and must stop / roll back

CONTINUE = "carry on"; EXIT = "stop this block".

🧪 Why the choice matters (the class scenario)

Imagine a script inserting **50 rows**, and row 3 is a duplicate.

- With a **CONTINUE** handler: the error on row 3 is handled, and rows 4 … 50 are still inserted. **One row skipped, the rest survive.**
- Without a handler (or with **EXIT**): the program stops at row 3 and rows 4 … 50 are never inserted.

The teacher noted: to make a **multi-row script** keep going after a bad row, write the statements into a script and give it a `CONTINUE` handler — otherwise MySQL stops at the first error.

📜 The condition types you can handle

Condition	Meaning	Typical use
Error code	a specific MySQL numeric error	`1062` (duplicate key)
SQLSTATE	a 5-character standard code	`23000` (constraint violation)
SQLWARNING	warning conditions	"should have been more precise" style warnings
NOT FOUND	no data / no rows found	**used with cursors**
SQLEXCEPTION	every SQL error *other than* `SQLWARNING` and `NOT FOUND`	a **general** error handler

💡 `SQLEXCEPTION` is the catch-all

`SQLEXCEPTION` is the MySQL analogue of a general `catch (Exception e)` — it catches *any* SQL error that is not a warning and not "not found". Use it when you want one safety net that covers everything.

🧪 Example — a friendly message for duplicate keys (`add_student`)

Business rule: *if a student ID already exists, tell the user in plain words instead of dumping* `ERROR 1062`.

```
DELIMITER $$

CREATE PROCEDURE add_student(
    IN SID      INT,
    IN SName    VARCHAR(30),
    IN SDeptID  INT,
    IN SMarks   DECIMAL(5,2)
)
BEGIN
    -- Catch the duplicate-primary-key error (1062) and print a message
    DECLARE CONTINUE HANDLER FOR 1062
        SELECT 'Student ID is already present' AS message;

    INSERT INTO student (student_id, name, department_id, marks)
    VALUES (SID, SName, SDeptID, SMarks);
END $$

DELIMITER ;
```

Run it

```
CALL add_student(102, 'Nishad', 10, 95.67);   -- 102 already exists
```

Expected output — a friendly message instead of the raw error:

```
+-------------------------------+
| message                       |
+-------------------------------+
| Student ID is already present  |
+-------------------------------+
```

```
CALL add_student(110, 'Snehal', 20, 80);      -- 110 is free -> inserted normally
```

> ℹ️ **The handler only fires when the condition matches**
>
> We declared the handler **for `1062` specifically**. A *different* error would not be caught by it — it would behave as an unhandled error. To catch everything, use `SQLEXCEPTION` (below).

> 💡 **About `IN` parameters**
>
> In the demo a student asked why `IN` is written for every parameter. The answer: `IN` **is the default mode** for parameters, so you may write it or leave it out. Writing `IN` explicitly is fine and self-documenting; `OUT` / `INOUT` must be written when you need them.

🏷️ Named conditions

A **named condition** gives a **meaningful name to an error** so your code — and the user — understand it. Instead of remembering `1062`, you name it.

```
DECLARE duplicate_student_id CONDITION FOR SQLSTATE '23000';
DECLARE CONTINUE HANDLER FOR duplicate_student_id
    SELECT 'Duplicate student ID — please use a different one' AS message;
```

The name `duplicate_student_id` is clearer than the number, and the handler now reads like English. This is the same idea behind messages such as "*student ID is already present*" — tell the user what went wrong in human terms.

📣 Raising your own error — `SIGNAL`

Sometimes **you** are the one who knows the data is invalid. `SIGNAL` is how you **raise your own error** from inside SQL.

```
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary cannot be negative';
```

- `SQLSTATE '45000'` is the conventional code for a **user-defined exception**.
- `SET MESSAGE_TEXT = …` is the message the user sees.

The class trigger (Day 8 revisited)

```
DELIMITER $$
CREATE TRIGGER trg_salary
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END $$
DELIMITER ;
```

```
INSERT INTO employee (emp_id, emp_name, salary, grade)
VALUES (107, 'Akshay', -20000, 'C');
```

```
ERROR 1644 (45000): Salary cannot be negative
```

> ℹ️ **Why this works so cleanly**
>
> The trigger is `BEFORE INSERT`, so the `SIGNAL` stops the row **before it is saved**. You get your own message — `ERROR 1644 (45000): Salary cannot be negative` — instead of a generic constraint error. (Recall: to *prevent* data, validate **BEFORE**.)

↩️ `RESIGNAL` — handle, act, then re-throw

`RESIGNAL` is used **inside a handler** when you want to catch an error, do some work (log it, clean up, roll back), and then **pass the error back to the caller** instead of swallowing it.

```
DELIMITER $$
CREATE PROCEDURE some_procedure()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        -- 1. perform some actions: log it / clean up / ROLLBACK ...
        -- 2. pass the ORIGINAL error back to the caller
        RESIGNAL;
    END;

    -- === normal operations / business logic ===
END $$
DELIMITER ;
```



RESIGNAL = "I dealt with it, but it is still an error — handle it upstream."

> 💡 **SIGNAL vs RESIGNAL**
>
> - `SIGNAL` raises a **new** error (you invent the message).
> - `RESIGNAL` **re-raises the current** error from inside a handler (after your extra work).

🌐 The general error handler (`SQLEXCEPTION`)

Earlier we named a specific code (`1062`). A **general error handler** uses `SQLEXCEPTION` to catch *any* SQL error and show one generic message.

```
DELIMITER $$
CREATE PROCEDURE general_demo()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        SELECT 'Some SQL error has occurred' AS message;

    -- ... your statements ...
END $$
DELIMITER ;
```

Approach	Catches	Benefit	Downside
`FOR 1062`	only duplicate keys	precise, helpful message	misses every other error
`FOR SQLEXCEPTION`	**everything**	one safety net, no codes to remember	message is vague; the user (and you) cannot tell *which* error

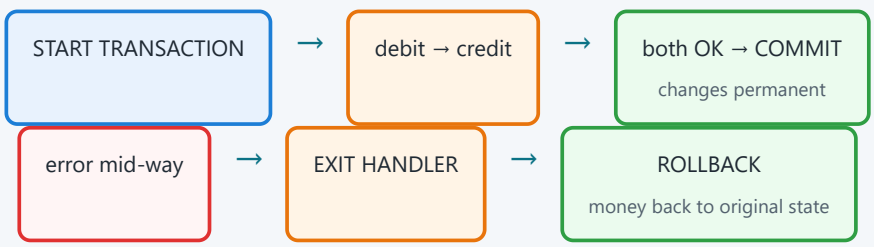
> ⚠️ **The trade-off the teacher highlighted**
>
> If you cannot remember error codes, `SQLEXCEPTION` with a generic message keeps the program from crashing — but it also **hides the specific cause**, so neither you nor the end user learns what actually went wrong. Use it when "something failed" is good enough; use a specific condition when you need a precise message.

💳 Transactions + error handling (roll back on failure)

This is the most important real-world use. Think of a **money transfer**: debit from one account and credit to another. If the debit succeeds but the credit fails, the money would vanish. The fix: wrap it in a **transaction** and **roll back** on any error.

```
DELIMITER $$
CREATE PROCEDURE transfer_money(
    IN from_acc INT,
    IN to_acc   INT,
    IN amount   DECIMAL(10,2)
)
BEGIN
    -- If ANY error happens, undo everything and report it
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Transaction failed — all changes rolled back' AS message;
    END;

    START TRANSACTION;
        UPDATE accounts SET balance = balance - amount WHERE account_id = from_acc;
        UPDATE accounts SET balance = balance + amount WHERE account_id = to_acc;
    COMMIT;
END $$
DELIMITER ;
```



Either the whole transfer happens, or none of it does — never half.

> ℹ️ **The teacher's account analogy**
>
> If money is debited from your friend's account but **not** credited to yours, the transaction must **roll back** so the friend's money is restored — you return to the earlier state. If both debit and credit succeed, `COMMIT` makes the changes permanent and no further change should happen.

🧩 Where you can use error handling

- **Stored procedures** — the examples above.
- **Stored functions** — same `DECLARE … HANDLER` syntax inside the function body.
- **Triggers** — e.g. the `SIGNAL`-based validation trigger.

In other words, **every stored program** can have handlers, and any of them can use the condition types and `SIGNAL` / `RESIGNAL` you learned here.

✅ Part B recap

- **Error handling** = detect an error and decide what the program does next.
- **Without a handler** the program stops; **with one**, it can continue, exit, or roll back.
- **MySQL error code** (`1062`) = one specific error; **SQLSTATE** (`23000`) = the category; **custom SQLSTATE** (`45000`) = your own message.
- Handler syntax: `DECLARE {CONTINUE|EXIT} HANDLER FOR <condition> <statement>;` — declared **inside the** `BEGIN … END` block.
- **CONTINUE** keeps going; **EXIT** leaves the current block.
- Condition types: **error code**, **SQLSTATE**, **SQLWARNING**, **NOT FOUND** (cursors), **SQLEXCEPTION** (general).
- `SIGNAL SQLSTATE '45000'` raises your own error; `RESIGNAL` re-raises the current one from inside a handler.
- **Transactions + error handler** → `ROLLBACK` so a half-done operation never survives.
- Handlers work in **procedures, functions and triggers**.

Now let's lock it all in. 🏁

Part C — Cheat Sheet, Gotchas & Practice 🏁

Everything from today, compressed into the pages you will actually come back to.

📋 Cheat sheet

Cursor — full skeleton

```
DELIMITER $$
CREATE PROCEDURE sp_name()
BEGIN
    DECLARE done     BOOLEAN DEFAULT FALSE;      -- 1. variables
    DECLARE v_a <type>;
    DECLARE v_b <type>;

    DECLARE cur CURSOR FOR                       -- 2. cursor
        SELECT col_a, col_b
          FROM your_table
         WHERE some_condition;

    DECLARE CONTINUE HANDLER FOR NOT FOUND       -- 3. handler
        SET done = TRUE;

    OPEN cur;

    my_loop: LOOP
        FETCH cur INTO v_a, v_b;
        IF done THEN
            LEAVE my_loop;
        END IF;

        -- === your row-by-row logic here ===
    END LOOP;

    CLOSE cur;
END $$
DELIMITER ;
```

- Declaration order is **variables → cursor → handler**.
- **Always `CLOSE` the cursor.**
- Loop alternatives: `WHILE NOT done DO … END WHILE` (fetch once before) or `REPEAT … UNTIL done END REPEAT`.

Handler — skeletons

```
-- Continue: handle the error, then carry on
DECLARE CONTINUE HANDLER FOR 1062
    SELECT 'Student ID is already present' AS message;

-- Exit: handle the error, then leave the current block
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SELECT 'Transaction failed' AS message;
END;
```

Raising / re-raising errors

```
-- SIGNAL: raise your OWN error
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary cannot be negative';

-- RESIGNAL: from inside a handler, pass the current error back up
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    -- log / cleanup / rollback ...
    RESIGNAL;
END;
```

The condition types

Condition	What it matches	Where it is used
`1062` (error code)	a specific MySQL error (duplicate key)	targeted handling
`SQLSTATE '23000'`	the integrity-violation category	standard-code handling
`SQLWARNING`	warnings	warning handling
`NOT FOUND`	no row / no data	**cursors**
`SQLEXCEPTION`	all SQL errors except the two above	general safety net
`SQLSTATE '45000'`	your own `SIGNAL`	user-defined errors

Cursor vs set-based — one more time

Prefer the **query** when you can	Prefer the **cursor** when you must
`UPDATE … WHERE`, `DELETE …`, `INSERT … SELECT`	different decision per row
`JOIN`, `GROUP BY`, aggregate functions	state depends on the previous row
CTEs (`WITH …`), window functions	progressively built report/string

Admin / inspection commands

Command	Shows
`SHOW PROCEDURE STATUS WHERE Db = DATABASE();`	your procedures
`SHOW CREATE PROCEDURE proc_name;`	a procedure's definition
`SHOW TRIGGERS;`	triggers in the database
`DROP PROCEDURE IF EXISTS proc_name;`	remove a procedure

🐞 Errors and gotchas we actually hit today

Error / problem	Why it happened	The fix
`ERROR 1062 (23000)` duplicate entry	inserting a primary key that already exists	handle it with `DECLARE CONTINUE HANDLER FOR 1062`, or use a fresh id
A multi-row insert stops at the first bad row	no handler (or an `EXIT` handler) — MySQL halts on error	give the script a `CONTINUE` handler so later rows still go in
Cursor loop never ends / loops one row too many	the `NOT FOUND` handler is missing, so `done` never flips	always declare `DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;`
Vague syntax error inside `BEGIN … END`	declarations are out of order	declare in order **variables → cursor → handler**
Cursor "does not exist" / unknown	it was declared at the top level instead of inside a stored program	declare and use it inside a procedure / function / trigger
Duplicates appearing twice while testing	you re-ran the increment/archive procedure on already-changed rows	reset the data (or guard with a `WHERE`) before re-running

Every one of these came up in today's session.

> ⚠️ **Re-running a cursor procedure changes data again**
>
> `apply_salary_increment` and `archive_low_salary` **do** change/insert rows. Running them twice applies the logic twice. During practice, note the state before you call, then re-seed the table if you want a clean run.

🧠 Practice exercises (do these — they mirror the class)

> ❓ **Exercise 1 — count the rows a cursor walks**
>
> Write a procedure `count_high_earners()` that uses a cursor over `employee` and sets an `INT` counter for every employee whose salary is **above** 50,000.

> ❓ **Exercise 2 — give a custom increment**
>
> Write a cursor that reads `emp_id` and `salary` and adds **8 %** to every employee earning **strictly between** 40,000 and 70,000.

❓ Exercise 3 — convert a cursor to set-based SQL

Rewrite the grade-based increment **without** a cursor, using a single `UPDATE … CASE …`. (Hint: `SET salary = salary * CASE grade WHEN 'A' THEN 1.05 …`)

❓ Exercise 4 — friendly duplicate message

Recreate `add_student`, but use a **named condition** for the duplicate error and a message that names the student ID.

❓ Exercise 5 — block zero marks

Write a `BEFORE INSERT` trigger on `student` that raises `SIGNAL SQLSTATE '45000'` with the message "*Marks must be greater than zero*" if `NEW.marks <= 0`.

❓ Exercise 6 — roll back a transfer

Write a procedure that moves 10,000 from account 1 to account 2 inside a transaction with an `EXIT` handler that `ROLLBACK`s on any error.

❓ Exercise 7 — general handler

Write a procedure with one `SQLEXCEPTION` handler that prints a generic message no matter which error occurs.

🧩 Solutions

1. `count_high_earners()`

```
DELIMITER $$
CREATE PROCEDURE count_high_earners()
BEGIN
    DECLARE done      BOOLEAN DEFAULT FALSE;
    DECLARE vSalary   DECIMAL(10,2);
    DECLARE high_cnt  INT DEFAULT 0;

    DECLARE cur CURSOR FOR SELECT salary FROM employee;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO vSalary;
        IF done THEN LEAVE read_loop; END IF;

        IF vSalary > 50000 THEN
            SET high_cnt = high_cnt + 1;
        END IF;
    END LOOP;
    CLOSE cur;

    SELECT high_cnt AS employees_above_50k;
END $$
DELIMITER ;
```

2. Custom 8 % increment (40,000 < salary < 70,000)

```
DELIMITER $$
CREATE PROCEDURE raise_mid_band()
BEGIN
    DECLARE done    BOOLEAN DEFAULT FALSE;
    DECLARE vEmpId  INT;
    DECLARE vSalary DECIMAL(10,2);

    DECLARE cur CURSOR FOR SELECT emp_id, salary FROM employee;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;
    pay_loop: LOOP
        FETCH cur INTO vEmpId, vSalary;
        IF done THEN LEAVE pay_loop; END IF;

        IF vSalary > 40000 AND vSalary < 70000 THEN
            UPDATE employee SET salary = salary * 1.08 WHERE emp_id = vEmpId;
        END IF;
    END LOOP;
    CLOSE cur;
END $$
DELIMITER ;
```

3. The same logic as one set-based `UPDATE`

```
UPDATE employee
   SET salary = salary * CASE grade
                           WHEN 'A' THEN 1.05
                           WHEN 'B' THEN 1.10
                           WHEN 'C' THEN 1.15
                           ELSE 1.13
                         END;
```

> 💡 **The lesson**
>
> When the decision depends only on **the row itself** (here, on `grade`), a `CASE`-based `UPDATE` does the whole job **faster** — this is exactly the "prefer set-based SQL" rule. A cursor is for decisions that truly need row-by-row procedural work.

4. `add_student` with a named condition

```
DELIMITER $$
CREATE PROCEDURE add_student2(
    IN SID INT, IN SName VARCHAR(30), IN SDeptID INT, IN SMarks DECIMAL(5,2)
)
BEGIN
    DECLARE duplicate_id CONDITION FOR SQLSTATE '23000';

    DECLARE CONTINUE HANDLER FOR duplicate_id
        SELECT CONCAT('Student ID ', SID, ' is already present') AS message;

    INSERT INTO student (student_id, name, department_id, marks)
    VALUES (SID, SName, SDeptID, SMarks);
END $$
DELIMITER ;
```

5. Block zero (or negative) marks

```
DELIMITER $$
CREATE TRIGGER trg_check_marks
BEFORE INSERT ON student
FOR EACH ROW
BEGIN
    IF NEW.marks <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Marks must be greater than zero';
    END IF;
END $$
DELIMITER ;
```

6. Transfer with rollback

```
DELIMITER $$
CREATE PROCEDURE transfer_money(IN from_acc INT, IN to_acc INT, IN amt DECIMAL(10,2))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Transaction failed — rolled back' AS message;
    END;

    START TRANSACTION;
        UPDATE accounts SET balance = balance - amt WHERE account_id = from_acc;
        UPDATE accounts SET balance = balance + amt WHERE account_id = to_acc;
    COMMIT;
END $$
DELIMITER ;
```

7. General error handler

```
DELIMITER $$
CREATE PROCEDURE general_demo()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        SELECT 'Some SQL error has occurred' AS message;

    -- ... statements that might fail ...
END $$
DELIMITER ;
```

🎓 Final words

Today you learned the last two MySQL topics — and both are about **control**:

- **Cursors** give you **row-level control**: read a row, decide, act. Powerful, but slow — so reach for set-based SQL first and use a cursor only when the logic truly is row-by-row.
- **Error handling** gives you **failure control**: with `CONTINUE` / `EXIT` handlers, `SIGNAL`, `RESIGNAL` and `ROLLBACK`, a program decides what happens when the world is not perfect instead of simply crashing.
- The `NOT FOUND` handler you used for cursors is the *same idea* as the `1062` handler you used for duplicate keys — one mechanism, many conditions.

> 💡 **Exam note (from class)**
>
> - **Cursors will NOT be asked in the lab exam** — but the teacher insisted you **practise them anyway**, because they strengthen how you reason about row-by-row logic.
> - **Error handling** was covered on the last day, so the chances of it appearing are lower; **read the theory** for the CMC/theory paper (conditions, SQLSTATE, handlers, `SIGNAL`).

End of Day 9 notes.

Part D — Exam Briefing & What's Next 📝

Vipul sir joined the session to give the final instructions for the **Monday MySQL exam**. These are not technical notes, but they matter as much as the code — read them once more the night before.

🗓️ The schedule (Monday)

Time	What happens
9:00 AM	Reporting time
9:30 AM onward	**MCQ** (theory)
10:00 – 12:00	**Lab exam** (practical, on MySQL command line)
12:00 – 12:15	Upload the solution
12:15 onward	**Spot evaluation** (in the same Zoom recorded environment)

- Tuesday is a **holiday**.
- The **WPT module** starts **21st** (Wednesday), running **7–9 AM** and **7–9 PM**.
- The pending topics (**Normalization**, **NoSQL**, **MongoDB**) are covered later; **MongoDB is NOT in this exam**.

🖥️ The environment rules (read carefully)

> 🛑 **Only three things may be open during the exam**
>
> 1. Your **question paper** (shared on your **mail ID**),
> 2. the **MySQL command line** (not Workbench!),
> 3. **one** Notepad file.
>
> Anything else must be closed. No online compilers — the exam must be done on your own system's MySQL command line / command prompt. **No breaks** during the exam.

- **Use the MySQL command line / command prompt** — `Workbench` **is NOT allowed** for the exam. There is a difference: you must reach the *MySQL* command prompt, not just a plain Windows `cmd`.
- Type your queries in **Notepad** so you can **edit** them, then **copy-paste** into the command line and execute.

> 💡 **Why Notepad, not Word?**
>
> Word's **smart quotes** differ from the command line's straight quotes, so a query pasted from Word can fail. Notepad keeps plain `'...'` / `"..."`. (If Word works for you after a test, it is allowed — but Notepad is the safe choice.)

- Create a **new database** for the exam (if the paper does not name one, use something like `dbt_exam`). **Do not** work in the database you practised in.
- You may use the `root` user, or create a **new user** and grant it permissions — your choice.

📄 The solution file — how marks are awarded

The solution is a **document (Word is fine)**. For **each question**:

1. Write the **question number**.
2. Paste the **query text** (in text form).
3. Paste a **full-screen screenshot of the output**.

> ⚠️ **Screenshot rules (0 marks if broken)**
>
> - The screenshot must be the **output of the command line**, in **full-screen** mode.
> - A screenshot that captures only a part of the screen will be **penalised**.
> - If a query is missing, or the screenshot is missing → **0 marks** for that question.
> - One query = one screenshot. (You *may* reuse a single full-screen screenshot across questions **only** if it genuinely shows those queries' output — but be safe and take a fresh, complete capture.)

Example of how to lay out each answer

```
Q1.
Query:
    CREATE DATABASE dbt_exam;
    USE dbt_exam;
    CREATE TABLE ...;
Output:
    [ full-screen screenshot of the command-line output ]
```

- After the exam you **upload the PDF** of your solution within the given window (**12:00 – 12:15**).
- **Spot evaluation** follows. If the command line is already closed by then, you explain from the **solution file** (query + screenshot).

> ℹ️ **Marks are per query**
>
> Each question — and each query — carries its own marks. There will be roughly **20–30 questions**, mixing one-liners with multi-step tasks (create tables, insert rows, joins, procedures, functions, triggers).

🧮 What the paper will ask (based on the briefing)

Everything must be done **from scratch** — including creating the database:

- **Create tables** and define **relationships** between them (foreign keys).
- `INSERT` / `UPDATE` / `DELETE` rows.
- `ALTER` / `DROP` commands.
- **Clauses**, **joins**, **subqueries**.
- **Stored procedures**, **functions**, **triggers** — each in its proper sequence.

> 💡 **The teacher's revision checklist (from the Q&A)**
>
> - **Constraints** (primary key, foreign key, unique, check) and the **cascades** (`ON DELETE CASCADE` , `ON UPDATE CASCADE` , `SET NULL`) — *important and used in industry*.
> - **Operators** and **predefined functions** — string, **date**, **number** functions (e.g. `LPAD` , `FLOOR` , `ROUND`); *expect a few questions here*.
> - **Window functions** — revise these for the theory/CMS paper.
> - **Joins and subqueries** — heavily used in industry; build them on 3–4 connected tables.
> - **Triggers** — remember to show **both** the "before" and "after" result screenshots so the trigger's effect is visible.

✅ Quick pre-exam checklist

- [ ] MySQL installed and the **command line** opens (not just Workbench).
- [ ] You know how to reach the **MySQL** command prompt.
- [ ] One fresh **Notepad** file ready for typing queries.
- [ ] Know the **full-screen screenshot** shortcut (`Win` + `PrtSc` / `Win` + `Shift` + `S`).
- [ ] Practice runs recorded in the **Zoom** environment.
- [ ] Revision done: constraints, cascades, operators, string/date/number functions, joins, subqueries, window functions, procedures, functions, triggers.
- [ ] Remember: **cursor is not asked** — but you practised it. 💪

> 💡 **The teacher's parting advice**
>
> Read the **question paper properly first** before solving. Create the database, then the tables and relationships, then the queries — step by step. **Practise as much as you can**; the paper mirrors the assignments you have already done.

Good luck! 🍀